

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Programación I

TRABAJO PRÁCTICO INTEGRADOR

Sistema de Gestión de Datos de Países

Integrantes

Rodolfo Ángel Gutiérrez

Lucía Daniela Gutiérrez

Año 2026

ÍNDICE

1. Introducción	3
2. Objetivos	3
3. Marco teórico	4
4. Desarrollo del sistema	6
5. Funcionalidades implementadas	8
6. Validaciones y manejo de errores	13
7. Dificultades encontradas	15
8. Conclusiones	16
9. Enlaces y bibliografía	17

1. Introducción

El presente Trabajo Práctico Integrador fue desarrollado en el marco de la asignatura Programación I de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional.

El proyecto consiste en el diseño e implementación de una aplicación de consola desarrollada en Python destinada a la gestión de datos de países. El sistema permite almacenar, consultar, modificar y administrar información relacionada con distintos países, utilizando como mecanismo de persistencia un archivo en formato CSV.

El sistema fue diseñado para ofrecer una interfaz sencilla mediante un menú interactivo, permitiendo al usuario gestionar la información de manera práctica y organizada.

Durante el desarrollo del proyecto se aplicaron los principales conceptos abordados en la materia, tales como programación estructurada, modularización mediante funciones, manejo de listas y diccionarios, validación de datos, tratamiento de excepciones mediante try/except y manipulación de archivos.

Asimismo, el trabajo permitió integrar diferentes técnicas de búsqueda, ordenamiento y procesamiento de información, promoviendo la resolución de problemas mediante el desarrollo de algoritmos y buenas prácticas de programación.

El objetivo principal del proyecto fue construir una aplicación funcional, robusta y sencilla de utilizar, aplicando los conocimientos adquiridos durante la cursada y fortaleciendo las habilidades de análisis, diseño e implementación de soluciones informáticas.

2. Objetivos

Objetivo general

Desarrollar una aplicación de consola en Python que permita gestionar información de distintos países mediante el uso de estructuras de datos, archivos CSV y técnicas

de programación estructurada, aplicando los conocimientos adquiridos durante la cursada de Programación I.

Objetivos específicos

- Implementar un sistema capaz de almacenar y administrar información sobre distintos países.
- Utilizar archivos CSV como mecanismo de persistencia de datos.
- Aplicar funciones para modularizar el código y facilitar su mantenimiento.
- Emplear listas y diccionarios como estructuras principales para el almacenamiento y procesamiento de la información.
- Incorporar validaciones y manejo de excepciones mediante try/except para mejorar la robustez del sistema.
- Implementar algoritmos de búsqueda, filtrado y ordenamiento de datos.
- Desarrollar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre los registros almacenados.
- Aplicar buenas prácticas de programación y documentación del código.

3. Marco teórico

El desarrollo del presente Trabajo Práctico Integrador se fundamenta en diversos conceptos abordados durante la cursada de Programación I, los cuales permitieron diseñar e implementar una solución informática capaz de gestionar información de manera organizada y eficiente.

Programación estructurada

La programación estructurada es un paradigma que propone dividir un problema complejo en partes más pequeñas y fáciles de resolver mediante estructuras

secuenciales, condicionales y repetitivas. Este enfoque favorece la claridad del código, facilita su mantenimiento y mejora la comprensión del programa.

En el presente proyecto, la programación estructurada se aplicó mediante la utilización de funciones específicas para cada una de las operaciones disponibles en el sistema.

Modularización mediante funciones

La modularización consiste en dividir un programa en funciones independientes, cada una encargada de realizar una tarea determinada. Esta metodología permite reutilizar código, simplificar las pruebas y facilitar futuras modificaciones.

El sistema desarrollado implementa funciones destinadas a la carga, búsqueda, actualización, eliminación, filtrado, ordenamiento y almacenamiento de la información.

Estructuras de datos

Las estructuras de datos permiten organizar la información para facilitar su procesamiento. En este proyecto se empleó una lista de diccionarios, donde cada diccionario representa un país y almacena sus principales características, tales como nombre, población, superficie y continente.

Esta estructura ofrece flexibilidad para agregar, modificar, buscar y eliminar registros de manera sencilla.

Archivos CSV

Los archivos CSV (Comma Separated Values) constituyen un formato ampliamente utilizado para el almacenamiento e intercambio de datos estructurados. Su principal ventaja radica en su simplicidad y compatibilidad con diversas aplicaciones.

En este proyecto se empleó un archivo CSV como mecanismo de persistencia de datos, permitiendo conservar la información almacenada entre distintas ejecuciones del programa.

Validación de datos y manejo de excepciones

La validación de datos tiene como objetivo garantizar que la información ingresada por el usuario sea correcta y consistente. Para ello, se implementaron diversas verificaciones relacionadas con datos vacíos, valores numéricos, nombres repetidos y continentes válidos.

Asimismo, se utilizó el manejo de excepciones mediante try/except para controlar errores comunes y evitar interrupciones inesperadas durante la ejecución del programa.

Algoritmos de búsqueda y ordenamiento

Los algoritmos de búsqueda permiten localizar información específica dentro de una estructura de datos, mientras que los algoritmos de ordenamiento organizan los registros según determinados criterios.

El sistema implementa mecanismos de búsqueda por nombre, filtrado por distintos atributos y ordenamiento de la información, facilitando el acceso y análisis de los datos almacenados.

Operaciones CRUD

El concepto CRUD representa las operaciones básicas que pueden realizarse sobre un conjunto de datos: Crear (Create), Leer (Read), Actualizar (Update) y Eliminar (Delete).

Durante el desarrollo del proyecto se implementaron estas operaciones, permitiendo agregar nuevos países, visualizar la información almacenada, modificar registros existentes y eliminar aquellos que ya no sean necesarios.

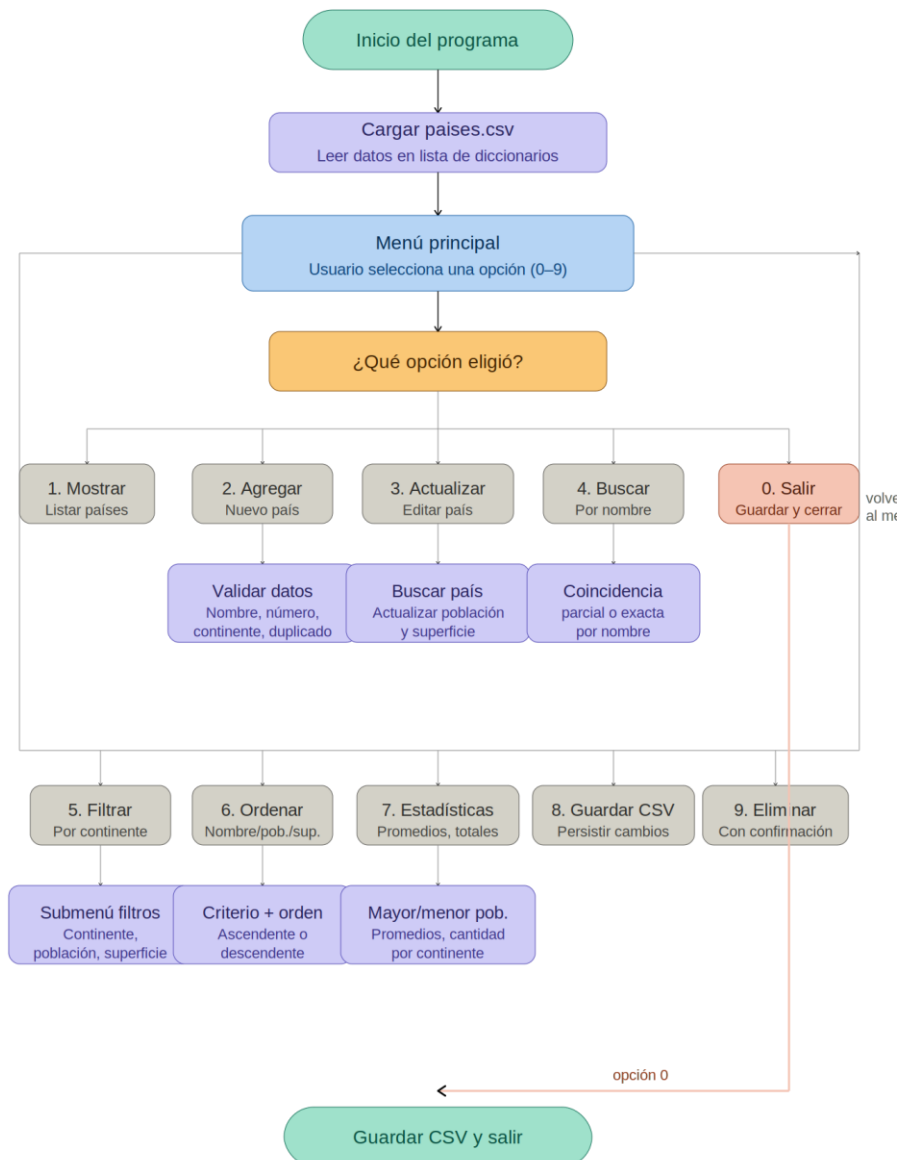
La aplicación integrada de estos conceptos permitió desarrollar un sistema funcional y robusto, aplicando de manera práctica los conocimientos adquiridos durante la asignatura Programación I.

4. Desarrollo del Sistema

El sistema fue desarrollado en Python como una aplicación de consola destinada a gestionar información de distintos países almacenada en un archivo CSV.

Para facilitar el mantenimiento del código y mejorar su legibilidad, el programa fue diseñado utilizando una estructura modular basada en funciones, donde cada una cumple una tarea específica dentro del sistema.

Diagrama de flujo



Estructura de los datos

La información se almacena en el archivo **paises.csv**, donde cada registro contiene los siguientes campos:

- Nombre del país.

- Población.
- Superficie.
- Continente.

Durante la ejecución del programa, los datos del archivo son cargados en memoria y manipulados mediante listas y diccionarios de Python, permitiendo realizar operaciones de consulta y modificación de manera eficiente.

5. Funcionalidades implementadas

El sistema desarrollado incorpora distintas funcionalidades que permiten administrar la información de los países almacenados en el archivo CSV de manera sencilla e intuitiva.

Las principales operaciones implementadas son las siguientes:

The screenshot shows a code editor with a Python script and its terminal output. The script defines functions for searching, loading, and managing a list of countries. The terminal output displays a menu of options and a detailed list of countries with their population, area, and continent.

```

def buscar_pais_en_lista(países, nombre):
    if normalizar_texto(país["nombre"]) == nombre_normalizado:
        return país
    return None

def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga países desde un archivo CSV.
    """

# Terminal Output:
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 1

--- LISTADO DE PAÍSES ---
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América
Nombre: Brasil | Población: 213993437 | Superficie: 8515767 km² | Continente: América
Nombre: Chile | Población: 19493184 | Superficie: 756102 km² | Continente: América
Nombre: Uruguay | Población: 3426800 | Superficie: 176215 km² | Continente: América
Nombre: México | Población: 126786138 | Superficie: 1964375 km² | Continente: América
Nombre: Estados Unidos | Población: 331900000 | Superficie: 9833517 km² | Continente: América
Nombre: Canadá | Población: 38930000 | Superficie: 9984670 km² | Continente: América
Nombre: Alemania | Población: 83149300 | Superficie: 357022 km² | Continente: Europa
Nombre: España | Población: 47420000 | Superficie: 505990 km² | Continente: Europa
Nombre: Francia | Población: 67750000 | Superficie: 551695 km² | Continente: Europa
Nombre: Italia | Población: 58200000 | Superficie: 301340 km² | Continente: Europa
Nombre: Reino Unido | Población: 67220000 | Superficie: 243610 km² | Continente: Europa
Nombre: Japón | Población: 125800000 | Superficie: 377975 km² | Continente: Asia
Nombre: China | Población: 1412000000 | Superficie: 9596961 km² | Continente: Asia
Nombre: India | Población: 1408000000 | Superficie: 3287263 km² | Continente: Asia
Nombre: Corea del Sur | Población: 51740000 | Superficie: 100210 km² | Continente: Asia
Nombre: Australia | Población: 25590000 | Superficie: 7692024 km² | Continente: Oceanía
Nombre: Nueva Zelanda | Población: 5123000 | Superficie: 268821 km² | Continente: Oceanía
Nombre: Sudáfrica | Población: 59890000 | Superficie: 1221037 km² | Continente: África
Nombre: Egipto | Población: 109300000 | Superficie: 1002450 km² | Continente: África
Nombre: Nigeria | Población: 213400000 | Superficie: 923768 km² | Continente: África
Nombre: Marruecos | Población: 37080000 | Superficie: 446550 km² | Continente: África
Nombre: Bolivia | Población: 2900000 | Superficie: 1080000 km² | Continente: América
  
```

- Mostrar países: permite visualizar el listado completo de los países almacenados en el sistema junto con sus principales datos.

```
def buscar_pais_en_lista(países, nombre):
    if normalizar_texto(pais["nombre"]) == nombre_normalizado:
        return pais
    return None

def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga países desde un archivo CSV.
    """
```

Nombre: Nigeria | Población: 213400000 | Superficie: 923768 km² | Continente: África
Nombre: Marruecos | Población: 37000000 | Superficie: 446550 km² | Continente: África
Nombre: Bolivia | Población: 2000000 | Superficie: 1000000 km² | Continente: América

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir

Seleccione una opción: 2

--- AGREGAR PAÍS ---
Ingrese nombre del país: Perú
Ingrese población: 45000000
Ingrese superficie en km²: 65000000
Ingrese continente: america
País agregado correctamente.

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir

Seleccione una opción:

- Agregar países: posibilita incorporar nuevos registros al archivo, verificando que la información ingresada sea válida.

```
def buscar_pais_en_lista(países, nombre):
    if normalizar_texto(pais["nombre"]) == nombre_normalizado:
        return pais
    return None

def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga países desde un archivo CSV.
    """
```

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir

Seleccione una opción: 3

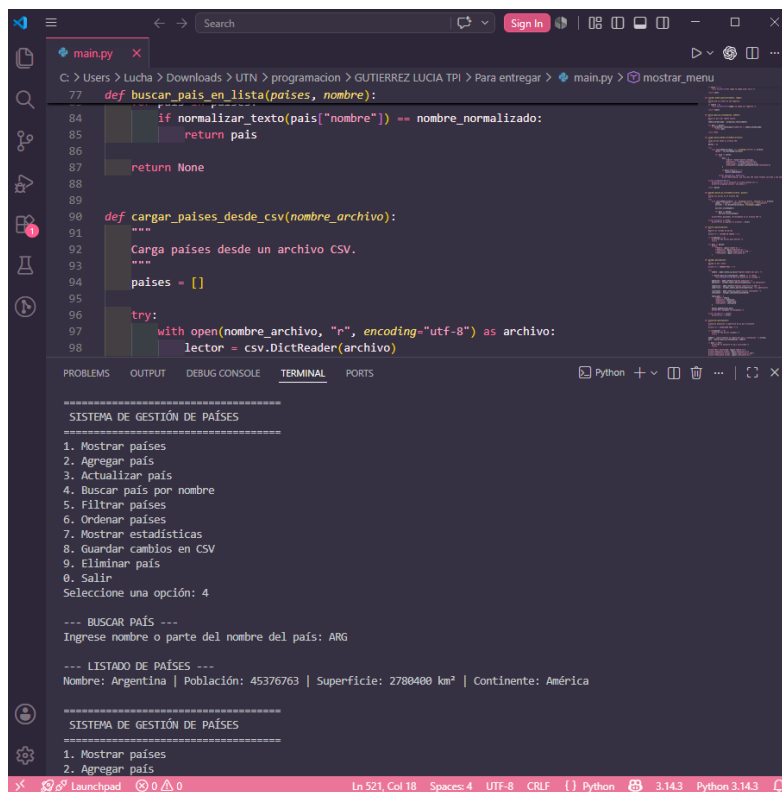
--- ACTUALIZAR PAÍS ---
Ingrese el nombre del país a actualizar: Perú
País encontrado: Perú
Población actual: 45000000
Superficie actual: 65000000 km²
Continente actual: América
Ingrese nueva población: 50000000
Ingrese nueva superficie en km²: 65000000
Ingrese nuevo continente: América
Datos actualizados correctamente.

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir

Seleccione una opción:

- Actualizar información: permite modificar los datos de un país previamente registrado.



```
def buscar_pais_en_lista(países, nombre):
    if normalizar_texto(pais["nombre"]) == nombre_normalizado:
        return pais
    return None

def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga países desde un archivo CSV.
    """
    países = []
    try:
        with open(nombre_archivo, "r", encoding="utf-8") as archivo:
            lector = csv.DictReader(archivo)
```

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 4

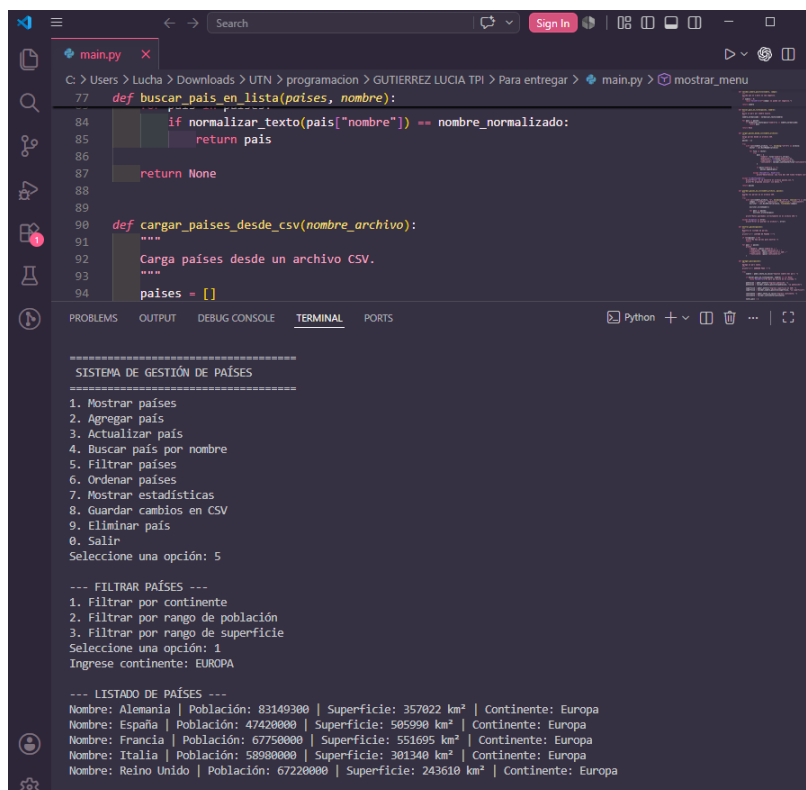
--- BUSCAR PAÍS ---
Ingrese nombre o parte del nombre del país: ARG

--- LISTADO DE PAÍSES ---
Nombre: Argentina | Población: 45376763 | Superficie: 2780480 km² | Continente: América

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país

- Buscar países: facilita localizar un país mediante su nombre, realizando las comparaciones correspondientes.



```
def buscar_pais_en_lista(países, nombre):
    if normalizar_texto(pais["nombre"]) == nombre_normalizado:
        return pais
    return None

def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga países desde un archivo CSV.
    """
    países = []
```

SISTEMA DE GESTIÓN DE PAÍSES

1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 5

--- FILTRAR PAÍSES ---
1. Filtrar por continente
2. Filtrar por rango de población
3. Filtrar por rango de superficie
Seleccione una opción: 1
Ingrese continente: EUROPA

--- LISTADO DE PAÍSES ---
Nombre: Alemania | Población: 83149300 | Superficie: 357022 km² | Continente: Europa
Nombre: España | Población: 47420000 | Superficie: 505990 km² | Continente: Europa
Nombre: Francia | Población: 67750000 | Superficie: 551695 km² | Continente: Europa
Nombre: Italia | Población: 58980000 | Superficie: 301340 km² | Continente: Europa
Nombre: Reino Unido | Población: 67220000 | Superficie: 243610 km² | Continente: Europa

- Filtrar información: permite obtener listados específicos según determinados criterios, como el continente al que pertenece el país.

```

main.py
C:\Users\> Lucha > Downloads > UTN > programacion > GUTIERREZ LUCIA TPI > Para entregar > main.py > mostrar_menu
77 def buscar_pais_en_lista(países, nombre):
84     if normalizar_texto(pais["nombre"]) == nombre_normalizado:
85         return pais
86     return None
87
88
89
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + Python 3.14.3 Python 3.14.3
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 6

--- ORDENAR PAÍSES ---
1. Ordenar por nombre
2. Ordenar por población
3. Ordenar por superficie
Seleccione criterio de ordenamiento: 1
1. Ascendente
2. Descendente
Seleccione tipo de orden: 1

--- LISTADO DE PAÍSES ---
Nombre: Alemania | Población: 83149300 | Superficie: 357022 km² | Continente: Europa
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América
Nombre: Australia | Población: 25690000 | Superficie: 7692024 km² | Continente: Oceanía
Nombre: Bolivia | Población: 2000000 | Superficie: 1000000 km² | Continente: América
Nombre: Brasil | Población: 213993437 | Superficie: 851767 km² | Continente: América
Nombre: Canadá | Población: 38930000 | Superficie: 9984670 km² | Continente: América
Nombre: Chile | Población: 19493104 | Superficie: 756102 km² | Continente: América
Nombre: China | Población: 1412000000 | Superficie: 9596961 km² | Continente: Asia
Nombre: Corea del Sur | Población: 51740000 | Superficie: 100210 km² | Continente: Asia
Nombre: Egipto | Población: 109300000 | Superficie: 1002450 km² | Continente: África
Nombre: España | Población: 47420000 | Superficie: 505990 km² | Continente: Europa
Nombre: Estados Unidos | Población: 331900000 | Superficie: 9833517 km² | Continente: América
Nombre: Francia | Población: 67750000 | Superficie: 551695 km² | Continente: Europa
Nombre: India | Población: 1400000000 | Superficie: 3287263 km² | Continente: Asia
Nombre: Italia | Población: 58900000 | Superficie: 301340 km² | Continente: Europa
Nombre: Japón | Población: 125800000 | Superficie: 377975 km² | Continente: Asia
Nombre: Marruecos | Población: 37000000 | Superficie: 446550 km² | Continente: África
Nombre: México | Población: 128705138 | Superficie: 1964375 km² | Continente: América
Nombre: Nigeria | Población: 213400000 | Superficie: 923760 km² | Continente: África
Nombre: Nueva Zelanda | Población: 5123000 | Superficie: 268021 km² | Continente: Oceanía
Nombre: Perú | Población: 50000000 | Superficie: 6500000 km² | Continente: América
Nombre: Reino Unido | Población: 67220000 | Superficie: 243610 km² | Continente: Europa
Nombre: Sudáfrica | Población: 59890000 | Superficie: 1221037 km² | Continente: África
Nombre: Uruguay | Población: 3426000 | Superficie: 176215 km² | Continente: América

```

- Ordenar registros: organiza la información almacenada según distintos criterios, facilitando su consulta y análisis.

```

main.py
C:\Users\> Lucha > Downloads > UTN > programacion > GUTIERREZ LUCIA TPI > Para entregar > main.py > mostrar_menu
77 def buscar_pais_en_lista(países, nombre):
84     if normalizar_texto(pais["nombre"]) == nombre_normalizado:
85         return pais
86     return None
87
88
89
90 def cargar_paises_desde_csv(nombre_archivo):
91     """
92     Carga países desde un archivo CSV.
93     """
94     países = []
95
96     try:
97         with open(nombre_archivo, "r", encoding="utf-8") as archivo:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + Python 3.14.3 Python 3.14.3

```

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 7

--- ESTADÍSTICAS ---
País con mayor población: China (1412000000)
País con menor población: Bolivia (2000000)
Promedio de población: 191848617.58
Promedio de superficie: 5286956.75 km²

Cantidad de países por continente:
América: 9
Europa: 5
Asia: 4
Oceanía: 2
África: 4
=====

```

- Mostrar estadísticas: presenta información general obtenida a partir de los datos almacenados en el sistema.

```

77 def buscar_pais_en_lista(paises, nombre):
84     if normalizar_texto(pais["nombre"]) == nombre_normalizado:
85         return pais
86     return None
87
88
89
90 def cargar_paises_desde_csv(nombre_archivo):
91     """
92     Carga paises desde un archivo CSV.
93     """
94     paises = []
95
96     try:
97         with open(nombre_archivo, "r", encoding="utf-8") as archivo:

```

América: 9
 Europa: 5
 Asia: 4
 Oceanía: 2
 África: 4

=====
 SISTEMA DE GESTIÓN DE PAÍSES
 =====
 1. Mostrar paises
 2. Agregar país
 3. Actualizar país
 4. Buscar país por nombre
 5. Filtrar paises
 6. Ordenar paises
 7. Mostrar estadísticas
 8. Guardar cambios en CSV
 9. Eliminar país
 0. Salir
 Seleccione una opción: 9

--- ELIMINAR PAÍS ---
 Ingrese el nombre del país a eliminar: Perú
 País encontrado: Perú | Población: 50000000 | Superficie: 65000000 km² | Continente: América
 ¿Confirma que desea eliminar este país? (S/N): s
 País eliminado correctamente.

=====
 SISTEMA DE GESTIÓN DE PAÍSES
 =====

- Eliminar países: permite quitar registros existentes cuando el usuario así lo requiera.

```

90 def cargar_paises_desde_csv(nombre_archivo):
91     """
92     Carga paises desde un archivo CSV.
93     """
94     paises = []
95
96     try:
97         with open(nombre_archivo, "r", encoding="utf-8") as archivo:

```

Seleccione una opción: 9

--- ELIMINAR PAÍS ---
 Ingrese el nombre del país a eliminar: Perú
 País encontrado: Perú | Población: 50000000 | Superficie: 65000000 km² | Continente: América
 ¿Confirma que desea eliminar este país? (S/N): s
 País eliminado correctamente.

=====
 SISTEMA DE GESTIÓN DE PAÍSES
 =====
 1. Mostrar paises
 2. Agregar país
 3. Actualizar país
 4. Buscar país por nombre
 5. Filtrar paises
 6. Ordenar paises
 7. Mostrar estadísticas
 8. Guardar cambios en CSV
 9. Eliminar país
 0. Salir
 Seleccione una opción: 8
 Datos guardados correctamente en el archivo CSV.

- Guardar cambios: almacena las modificaciones realizadas en el archivo CSV, garantizando la persistencia de los datos para futuras ejecuciones del programa.

Todas estas funcionalidades se encuentran integradas mediante un menú interactivo, permitiendo que el usuario acceda a cada una de ellas de manera sencilla y organizada durante la ejecución del sistema.

6. Validaciones y manejo de errores

Con el objetivo de mejorar la confiabilidad del sistema y evitar errores durante su utilización, se incorporaron distintas validaciones y mecanismos de control de excepciones.

Estas verificaciones permiten asegurar que los datos ingresados por el usuario sean consistentes y que el programa continúe funcionando correctamente ante situaciones inesperadas.

Entre las principales validaciones implementadas se encuentran:



```
54
55 def pedir_texto_no_vacio(mensaje):
56     """
57     Solicita texto y valida que no esté vacío.
58     """
59     texto = input(mensaje).strip()
60
61     if texto == "":
62         raise ValueError("El campo no puede estar vacío.")
63
64     return texto
65
66
```

```
=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 2

--- AGREGAR PAÍS ---
Ingrese nombre del país:
Error: El campo no puede estar vacío.

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
```

- Control de campos obligatorios, evitando el ingreso de datos vacíos.

```

=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 2

--- AGREGAR PAÍS ---
Ingrese nombre del país: Perú
Ingrese población: 4500000
Ingrese superficie en km²: a
Error: Debe ingresar un número entero válido.

```

- Validación de datos numéricos para los campos correspondientes a población y superficie.

```

177 superficie = validar_numero_positivo(superficie, "La superficie")
178
=====
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 4

--- BUSCAR PAÍS ---
Ingrese nombre o parte del nombre del país: ArgentInA

--- LISTADO DE PAÍSES ---
Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: América

```

- Comparación de nombres sin distinguir mayúsculas y minúsculas, facilitando las búsquedas y evitando inconsistencias.

```

21 def validar_continente(continente):
22     """
23     Valida que el continente ingresado sea correcto.
24     """
25     continentes_validos = ["america", "américa", "europa", "asia", "africa", "áfrica", "oceania", "oceanía"]
26
27     if normalizar_texto(continente) not in continentes_validos:
28         raise ValueError("Continente inválido. Debe ser: América, Europa, Asia, África u Oceanía.")
29
30     continente_normalizado = normalizar_texto(continente)
31
32     if continente_normalizado in ["america", "américa"]:
33         return "América"
34     elif continente_normalizado == "europa":
35         return "Europa"
36     elif continente_normalizado == "asia":
37         return "Asia"
38     elif continente_normalizado in ["africa", "áfrica"]:
39         return "África"
40     else:
41         return "Oceanía"
42
43
44 def pedir_entero(mensaje):
45     """
46     Solicita un número entero al usuario.
47     """
48     try:
49         return int(input(mensaje))

```

- Validación de continentes permitidos para garantizar la integridad de la información almacenada.



```
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 9

--- ELIMINAR PAÍS ---
Ingrese el nombre del país a eliminar: Perú
No se encontró el país solicitado.
```

```
SISTEMA DE GESTIÓN DE PAÍSES
=====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país por nombre
5. Filtrar países
6. Ordenar países
7. Mostrar estadísticas
8. Guardar cambios en CSV
9. Eliminar país
0. Salir
Seleccione una opción: 9

--- ELIMINAR PAÍS ---
Ingrese el nombre del país a eliminar: China
País encontrado: China | Población: 1412000000 | Superficie: 9596961 km² | Continente: Asia
¿Confirma que desea eliminar este país? (S/N): n
Operación cancelada.
```

- Verificación de la existencia de un país antes de realizar operaciones de actualización o eliminación.



```
def cargar_paises_desde_csv(nombre_archivo):
    """
    Carga paises desde un archivo CSV.
    """
    paises = []

    try:
        with open(nombre_archivo, "r", encoding="utf-8") as archivo:
            lector = csv.DictReader(archivo)

            for fila in lector:
                try:
                    pais = {
                        "nombre": fila["nombre"].strip(),
                        "poblacion": int(fila["poblacion"]),
                        "superficie": int(fila["superficie"]),
                        "continente": validar_continente(fila["continente"])
                    }

                    if pais["nombre"] != "":
                        paises.append(pais)

                except (ValueError, KeyError):
                    print("Advertencia: una fila del CSV tiene formato inválido y fue omitida.")

            except FileNotFoundError:
                print("Error: no se encontró el archivo paises.csv.")
                print("El programa iniciará sin datos.")

    return paises
```

- Control de errores durante la lectura y escritura del archivo CSV.

Asimismo, se implementó el manejo de excepciones mediante estructuras try/except, permitiendo detectar errores de ingreso de datos o problemas relacionados con los archivos sin interrumpir la ejecución del programa.

La incorporación de estas validaciones y controles contribuyó a mejorar la estabilidad del sistema y a ofrecer una experiencia de uso más segura y confiable para el usuario.

7. Dificultades encontradas

Durante el desarrollo del proyecto se presentaron distintos desafíos técnicos propios del proceso de programación, los cuales fueron abordados mediante pruebas, análisis y correcciones sucesivas.

Uno de los aspectos que requirió mayor atención fue el manejo de los datos almacenados en el archivo CSV, ya que resultó necesario controlar la correcta lectura

y escritura de la información para garantizar la persistencia de los registros entre distintas ejecuciones del programa.

Otro punto importante fue la implementación de validaciones, incorporando controles para evitar el ingreso de datos incorrectos y asegurar la consistencia de la información almacenada. Asimismo, fue necesario verificar el funcionamiento de las operaciones de búsqueda, actualización y eliminación de registros.

Durante el proceso de desarrollo también se realizaron diversas pruebas sobre las funcionalidades implementadas, detectando y corrigiendo pequeños errores que permitieron mejorar el comportamiento general del sistema y aumentar su estabilidad.

Estas dificultades formaron parte del proceso normal de desarrollo del proyecto y contribuyeron a fortalecer la comprensión de los contenidos trabajados durante la asignatura, destacando la importancia de la planificación, las pruebas de funcionamiento y la revisión continua del código.

8. Conclusiones

La realización de este Trabajo Práctico Integrador permitió aplicar e integrar los principales conceptos desarrollados durante la cursada de Programación I en un proyecto práctico y funcional.

A lo largo del desarrollo del sistema fue posible poner en práctica el uso de funciones, estructuras de control, listas, manejo de archivos CSV, validaciones, tratamiento de excepciones y algoritmos básicos de búsqueda y ordenamiento, comprendiendo la importancia de cada uno de estos elementos dentro de una aplicación real.

El proceso de desarrollo también permitió valorar la necesidad de realizar pruebas constantes y corregir errores a medida que surgían, contribuyendo a mejorar el funcionamiento del programa y a fortalecer las habilidades de análisis y resolución de problemas.

Asimismo, el trabajo en equipo favoreció el intercambio de ideas y la búsqueda conjunta de soluciones, tanto en el desarrollo del sistema como en la elaboración de la documentación correspondiente.

Consideramos que el proyecto cumplió con los objetivos planteados y representó una experiencia de aprendizaje significativa, permitiéndonos consolidar los conocimientos adquiridos durante la asignatura y comprender mejor las distintas etapas involucradas en el desarrollo de una aplicación informática.

Enlaces:

- Video explicativo: <https://www.youtube.com/watch?v=a7OZpZcHuJo>
- Github: https://github.com/rodolfogutierrez0835/TPI_Programacion_I/tree/main

9. Bibliografía

- Python Software Foundation. (2026). *Python Documentation*. <https://docs.python.org/3/>
- Python Software Foundation. (2026). *csv — CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>
- Microsoft. (2026). *Visual Studio Code Documentation*. <https://code.visualstudio.com/docs>
- Universidad Tecnológica Nacional. (2026). *Material de estudio de Programación I*. Tecnicatura Universitaria en Programación.